CS498: Algorithmic Engineering. Final Project (30%)

Overview

The final project is your chance to go deep. Pick a topic connected to what we have covered in class (or will cover) – LP/IP solvers, convex optimization, metaheuristics, SAT/SMT, or Machine Learning Systems – and either **implement**, **benchmark**, or **extend** something substantial. The goal is to practice the engineering side of algorithms: build a system, measure it, compare it to baselines, and write it up clearly.

This is an **individual** project.

Project Tracks

Choose one of the following three directions.

Track A: Reproduce a Research Paper (and potentially extend it).

Pick a paper from a venue like ALENEX, SEA, Mathematical Programming Computation, INFORMS Journal on Computing, NeurIPS, ICML, or similar. Reproduce the main algorithm, implement one or more baselines, and compare performance (speed, solution quality, or both) on benchmark instances.

What we are looking for: faithful implementation, fair baselines, clear experimental methodology, and honest discussion of results (including when the paper's method does *not* win).

If you find an existing open-source implementation of the paper's algorithm, reproduce (some) of its main results, then (optionally) extend it in a meaningful way: new datasets, an algorithmic tweak, new baseline/heuristic to compare against, a different solver backend, or a head-to-head comparison the original authors did not include.

For example, one idea of a project is you can try implementing the algorithm from this paper, and benchmarking it against vanilla KNN. Adding new test instances, and measuring more than just the speed (for example memory consumption of different algorithms, and various heuristics you can think of).

Track B: Deep Dive into a Course Topic

Take a topic we covered in class and go deeper. Read recent papers and heuristics, implement them, and present a comparative study. Examples:

- **Branching in Branch-and-Bound.** How do modern solvers decide which variable to branch on? Compare reliability branching, strong branching, pseudocost branching, and ML-based approaches on MIPLIB instances.
- **Cutting Planes.** Which cuts help most in practice? Implement Gomory mixed-integer cuts, compare cut selection strategies (efficacy, directed cutoff distance), and measure impact on integrality gap closure.
- **SAT Solver Heuristics.** Implement and compare VSIDS vs. Learning Rate Branching (LRB), or study the impact of inprocessing techniques (bounded variable elimination, vivification) on SAT competition benchmarks.
- **TSP Engineering.** Implement 2-opt/3-opt local search, compare against LKH and Concorde on TSPLIB instances, and study the effect of candidate edge generation.
- **ML and LLM Engineering.** Solve a problem using ML techniques, or use the LLM component of the course to build something cool.

One example here of a project is you could write a report surveying the various branching techniques used in modern branch-and-bound solvers, then benchmark them on open-source test instances to see how quickly they converge across different inputs, and optionally analyze which techniques come out on top.

Track C: Solve a New Problem

Use one of the methods from class (or a related method) to solve a problem that interests you. Model it, solve it, and compare approaches. Examples:

- Model a real scheduling, routing, or resource allocation problem as an IP and solve it with Gurobi.
- Encode a puzzle or verification task in SAT/SMT (or better create a new puzzle and its solver!) and compare different encodings (Bool vs. Int, different symmetry-breaking strategies).
- Apply convex optimization or SDP relaxations to a machine learning or signal processing problem.
- Use metaheuristics (Simulated Annealing, Genetic Algorithms) on a combinatorial problem and benchmark against exact solvers.

This track is the most open-ended. The key is having a clear problem, a clear method, and a clear comparison.

One example I'm personally excited about is writing a solver for Cities: Skylines (a game). My spouse is obsessed with this game, and we've both been curious about what a solver would look like. The combinatorial explosion of the action space makes exact solving virtually impossible, so it

would be fascinating to design a heuristic that can actually play the game! Alternatively, you could study a scheduling problem you encounter in real life, similar to what Fleetline does, and write a custom solver for it. It's really up to you!

Deliverables

1. Proposal (1 page max)

Due: **April 3 2026**

A **short** document covering:

- **Problem.** What are you working on and why is it interesting?
- **Approach.** Which track? What will you implement or compare?
- **Baselines.** What will you compare against?
- **Data.** What benchmarks or datasets will you use?
- **Timeline.** Rough plan for the remaining weeks.

You don't have to have everything ready; just a solid-ish plan. Things can go wrong (it's okay!), just keep us updated if your plan changes.

Submit as a PDF on Gradescope.

2. Final Submission

Due: **May 18 2026**

Two components:

Report (5 pages max, PDF). Use any reasonable format (LaTeX preferred). The report should include:

- **Introduction.** What problem are you solving and why?
- **Background.** Brief summary of the relevant method(s) from class or the paper you are implementing.
- **Method.** What did you build? Describe your implementation, any design choices, and any deviations from the original paper (if applicable).

- **Experiments.** Compare your method to baselines. Include tables or plots of runtime, solution quality, or other relevant metrics. Describe your experimental setup (hardware, solver versions, instance sizes).
- **Discussion.** What worked? What did not? What surprised you? Be honest.
- **References.**

Code (GitHub link, optional but strongly encouraged). If you implemented new algorithms or ran experiments:

- Include a `README.md` explaining how to navigate the code and reproduce results.
- Code should be clean enough for someone else to run.

Submit the report on Gradescope. Include the GitHub link in the report.

Suggested Papers and Topics

Below are starting points organized by course topic. You are welcome to choose papers not on this list (especially if you go through Track C!).

Branch-and-Bound and IP Solver Engineering

- Achterberg and Wunderling, *Mixed Integer Programming: Analyzing 12 Years of Progress*, Facets of Combinatorial Optimization, Springer, 2013. Ablation study of CPLEX's solver components (branching, cutting planes, presolve, heuristics).

- Gamrath et al., *Measuring the Impact of Branching Rules for Mixed-Integer Programming*, Operations Research Proceedings, 2018. Compares strong branching, reliability branching, pseudocost branching, and random branching on MIPLIB instances. Proposes a new evaluation measure (the "fair node number") that disentangles tree-size reductions from side-effects of branching rules. Clean experimental setup; You could reproduce the comparison on a subset of MIPLIB.

- Lodi and Tramontani, *Performance Variability in Mixed-Integer Programming*, INFORMS TutORials in Operations Research, 2013. Shows that changing random seeds, permuting rows/columns, or switching platforms causes large variability in MIP solver performance. You could replicate by running Gurobi with different seeds on benchmark instances and analyzing the resulting distributions.

- Achterberg et al., *Presolve Reductions in Mixed Integer Programming*, INFORMS Journal on Computing, 32(2), 2020. Comprehensive taxonomy of presolve techniques in Gurobi: bound tightening, coefficient reduction, probing, clique detection. You could implement a subset (e.g., bound tightening, singleton removal) and measure their impact.

- Delling et al., *Exact Combinatorial Branch-and-Bound for Graph Bisection*, ALENEX 2012. Fully combinatorial B&B for minimum graph bisection with improved bounds, branching rules, and graph contraction. Solves real-world graphs with millions of vertices.

- Akiba and Iwata, *Branch-and-Reduce Exponential/FPT Algorithms in Practice: A Case Study of Vertex Cover*, ALENEX 2015. Compares branch-and-reduce algorithms against ILP solvers (CPLEX) for minimum vertex cover. You can implement reduction rules and branching, then compare against Gurobi.

Cutting Planes

- Fischetti and Salvagnin, *A Relax-and-Cut Framework for Gomory Mixed-Integer Cuts*, Mathematical Programming Computation, 3(2), 2011. Uses a Lagrangian relax-and-cut scheme instead of adding Gomory cuts directly to the LP. Avoids numerical instability and improves cut quality. You could implement rank-1 GMI cut generation and compare the two approaches.

- Cook et al., *Numerically Safe Gomory Mixed-Integer Cuts*, INFORMS Journal on Computing, 21(4), 2009. Addresses numerical instability in Gomory cut generation. Proposes safe rounding procedures and coefficient tests. You could implement both naive and numerically safe GMI cuts and compare on ill-conditioned instances from MIPLIB.

- Wesselmann and Suhl, *Implementing Cutting Plane Management and Selection Techniques*, Technical Report, University of Paderborn, 2012. Practical guide to cut management: scoring metrics (efficacy, directed cutoff distance, objective parallelism), filtering, aging, and pool management. Focuses on the plumbing that textbooks skip.

TSP

- Helsgaun, *General k-opt Submoves for the Lin-Kernighan TSP Heuristic*, Mathematical Programming Computation, 1(2-3), 2009. Engineering of LKH-2: general k-opt moves, candidate edge generation via alpha-nearness, partitioning strategies. Experiments on instances up to 10M cities. You could implement 2-opt/3-opt heuristics for TSP and benchmark against LKH on TSPLIB.

- Cook et al., *Local Elimination in the Traveling Salesman Problem*, Mathematical Programming Computation, 16(4), 2024. Prunes edges from TSP instances using local k-opt moves and LP reduced-cost fixing. Tested on instances up to 115K cities. Combines exact and heuristic

techniques.

- Levine, *Finding the Right Cutting Planes for the TSP*, ALENEX 1999 / JEA. Investigates how to choose among violated cutting plane inequalities (subtour elimination, comb inequalities) in branch-and-cut for TSP. Directly relevant to lazy-constraint and user-cut approaches.

- Cirasella et al., *The Asymmetric TSP: Algorithms, Instance Generators, and Tests*, ALENEX 2001. Experimental comparison of ATSP heuristics: nearest neighbor, greedy, local search, cycle-patching. Introduces new instance generators. You can reproduce the comparison framework.

Network Flow and Min-Cut

- Henzinger et al., *Practical Minimum Cut Algorithms*, ALENEX 2018 / JEA. Near-linear-time algorithm for minimum cuts using label propagation contraction and Padberg-Rinaldi heuristics. Sequential and parallel implementations. Significantly faster than Stoer-Wagner on real-world graphs.

- Babenko et al., *Experimental Evaluation of Parametric Max-Flow Algorithms*, WEA 2007. Comparison of push-relabel variants for parametric maximum flow. Tests on real-world and adversarial instances. You can implement and compare standard max-flow algorithms with parametric extensions.

LP Solver Engineering

- Huangfu and Hall, *Parallelizing the Dual Revised Simplex Method*, Mathematical Programming Computation, 10(1), 2018. Two parallel dual simplex implementations in HiGHS. You could study the open-source HiGHS code and benchmark single-threaded vs. multi-threaded dual simplex.

MIP Modeling and Formulation

- Vielma, *Mixed Integer Linear Programming Formulation Techniques*, SIAM Review, 57(1), 2015. Survey of formulation techniques: extended formulations, ideal formulations, disjunctive programming, symmetry. You could take a problem (lot-sizing, facility location) and compare formulations.

- Bonami et al., *On Mathematical Programming with Indicator Constraints*, Mathematical Programming, 151(1), 2015. Indicator constraints vs. Big-M from both theoretical and computational perspectives. You could model the same problem both ways and compare solver performance.

- Pfetsch and Rehn, *A Computational Comparison of Symmetry Handling Methods for Mixed Integer Programs*, Mathematical Programming Computation, 11(1), 2019. Compares six symmetry handling methods (orbital fixing, orbital branching, symmetry-breaking inequalities, etc.) within SCIP on MIPLIB instances.

SDP and Max-Cut

- Mirka and Williamson, *An Experimental Evaluation of Semidefinite Programming and Spectral Algorithms for Max Cut*, SEA 2022. Implements and compares the Goemans-Williamson SDP 0.878-approximation against spectral algorithms for Max-Cut. Finds spectral algorithms are much faster and nearly as good in practice. Ideal for reproducing the SDP vs. spectral comparison.

- Ferizovic et al., *Engineering Kernelization for Maximum Cut*, ALENEX 2020. Data reduction rules (kernelization) for Max-Cut that achieve speedups of multiple orders of magnitude. You can implement the reduction rules and measure their effect.

Convex Optimization Engineering

- Stellato et al., *OSQP: An Operator Splitting Solver for Quadratic Programs*, Mathematical Programming Computation, 12(4), 2020. Design of OSQP, a first-order ADMM-based QP solver. Covers warm-starting, infeasibility detection, parameter selection. Benchmarked against Gurobi and MOSEK on 1,400 problems.

- There are many, many problems that can be solved using convex optimization. Pick your favorite and implement a convex programming solution to it, and compare it against heuristics.

SAT/SMT Engineering

- Biere and Fleury, *CaDiCaL 2.0*, CAV 2024. Engineering of the CaDiCaL solver family (SAT Competition winners 2020-2024). Covers chronological backtracking, vivification, bounded variable addition. You could study the open-source codebase and benchmark specific techniques (e.g., vivification on/off).

- Toda and Soh, *Implementing Efficient All Solutions SAT Solvers*, Journal of Experimental Algorithmics (JEA), Vol. 21, 2016. Techniques for AllSAT solvers: blocking clauses, clause-learning adaptations, BDD-based solution representation. You can implement an AllSAT solver building on a basic CDCL engine.

- Liang et al., *Learning Rate Based Branching Heuristic for SAT Solvers*, SAT 2016. Introduces LRB, a bandit-inspired variable selection heuristic for CDCL solvers. Implemented in MapleSAT (SAT Competition winner 2016/2018). You can directly compare against VSIDS approach for

example.

Metaheuristics and Local Search

- Henzinger et al., *ILP-based Local Search for Graph Partitioning*, SEA 2018. Uses small ILP subproblems (solved optimally) as local search moves to improve graph partitions. Clean demonstration of combining ILP solvers with metaheuristic frameworks.

- Grossmann et al., *Concurrent Iterated Local Search for Maximum Weight Independent Set*, SEA 2025. Concurrent hybrid iterated local search heuristic. Combines local search with parallel multicore execution. You can implement the sequential core and benchmark it.

- Langedal, Hespe, and Sanders, *Targeted Branching for the Maximum Independent Set Problem Using Graph Neural Networks*, SEA 2024. Combines exact branch-and-reduce with GNN-guided branching decisions. Bridges ML and combinatorial solving.

ML + Combinatorial Optimization

- Gasse et al., *Exact Combinatorial Optimization with Graph Convolutional Neural Networks*, NeurIPS 2019. Frames B&B variable selection as graph classification on the bipartite LP relaxation graph. Trained via imitation learning on strong branching. Public code available. The foundational “learning to branch” paper.

- Kool et al., *Attention, Learn to Solve Routing Problems!*, ICLR 2019. Transformer-based encoder-decoder for TSP and CVRP using REINFORCE. Baselines: Concorde, LKH3, OR-Tools. Public PyTorch code. One of the most reproduced papers in neural combinatorial optimization.

- Paulus et al., *Learning To Cut By Looking Ahead: Cutting Plane Selection via Imitation Learning*, ICML 2022. ML-based selection of which cutting planes to add during branch-and-cut. Uses lookahead policy as expert. You can implement this using Gurobi’s cut callback API and adapt this.

- Wu et al., *Learning Large Neighborhood Search Policy for Integer Programming*, NeurIPS 2021. ML model selects which part of the solution to destroy; Gurobi solves the repair subproblem exactly. A natural hybrid of PyTorch and Gurobi.

- Cappart et al., *Combining Reinforcement Learning and Constraint Programming for Combinatorial Optimization*, AAAI 2021. RL agent guides search within a constraint programming solver. Evaluated on TSP, graph coloring, and portfolio optimization. Comparison against pure CP, pure RL, and OR-Tools.

- Bengio et al., *Machine Learning for Combinatorial Optimization: A Methodological Tour d'Horizon*, European Journal of Operational Research, 2021. Comprehensive survey of the entire ML+CO field. Covers learning to branch, learning to cut, end-to-end neural solvers, algorithm selection. Good starting point for finding a project in this space.

Grading Rubric

Component	Weight	What we look for
Problem selection and motivation	10%	Clear, well-scoped, connected to course material
Technical depth	30%	Correct implementation, understanding of the method
Experimental evaluation	30%	Fair baselines, meaningful metrics, sufficient instances
Writing and presentation	20%	Clear writing, good figures/tables, honest discussion
Code quality (if applicable)	10%	Reproducible, documented, readable

Tips

- **Start early.** Getting baselines running always takes longer than expected.
- **Scope carefully.** A focused comparison on 3 methods and 5 benchmark instances is better than a vague attempt at 10 methods.
- **Be honest.** If your method is slower than the baseline, say so and explain why. Negative results are fine if well-analyzed.
- **Use what you know.** You have Gurobi, Z3, PyTorch, and Python. Leverage them.
- **Talk to us.** Come to office hours with your proposal idea. We can help you scope it.

Questions?

Come to office hours or post on the course forum.